Устройство сервера

Go-бэкенд ПОДБОР.про: из чего состоит, как проходит запрос, где правила. Версия после сокращения (октябрь 2026): 919 строк рабочего кода, 12 файлов, без комментариев. Все термины объяснены в разделе 14 «Словарь терминов».

1. Коротко

- Один исполняемый файл: REST-интерфейс `/api/v1` и раздача собранной клиентской части (`web/dist`).
- Договор — `server/api/openapi.yaml`. Он встроен в программу; каждый адрес из него отвечает: либо настоящим обработчиком, либо заглушкой с примером из описания.
- База — PostgreSQL (образ pgvector). Изменения схемы применяются сами при запуске (goose).
- Вход — JWT (HS256) на 24 часа, пароли — bcrypt.
- Сейчас настоящие обработчики у 5 адресов: `GET /health`, `POST /auth/register`, `POST /auth/login`, `GET /me`, `PATCH /me`. Остальные 19 — заглушки.

2. Запуск и настройки

Переменная	Что	По умолчанию
`DATABASE_URL`	адрес PostgreSQL	обязательно
`JWT_SECRET`	секрет подписи токенов, от 32 символов	обязательно
`HTTP_ADDR`	где слушать	`:8080`
`WEB_DIR`	папка со сборкой клиентской части	`./web/dist`

```
docker compose up --build        # вся система: db + ml + server, http://localhost:8080
docker compose up -d db          # только база, для разработки
cd server
make run                         # сервер с настройками для разработки
make test                        # тесты без базы (тесты с базой пропускаются)
make test-db                     # все тесты; ВНИМАНИЕ: очищают таблицу users
make generate                    # типы Go из openapi.yaml (make run/test делают это сами)
go run ./cmd/server -migrate-only   # только применить схему и выйти
```

`api/types.gen.go` в git не хранится: его создаёт `make generate` (и Dockerfile, и CI перед сборкой). Свежий клон без него не соберётся, IDE будет подсвечивать ошибки, пока не выполнить `make generate` один раз.

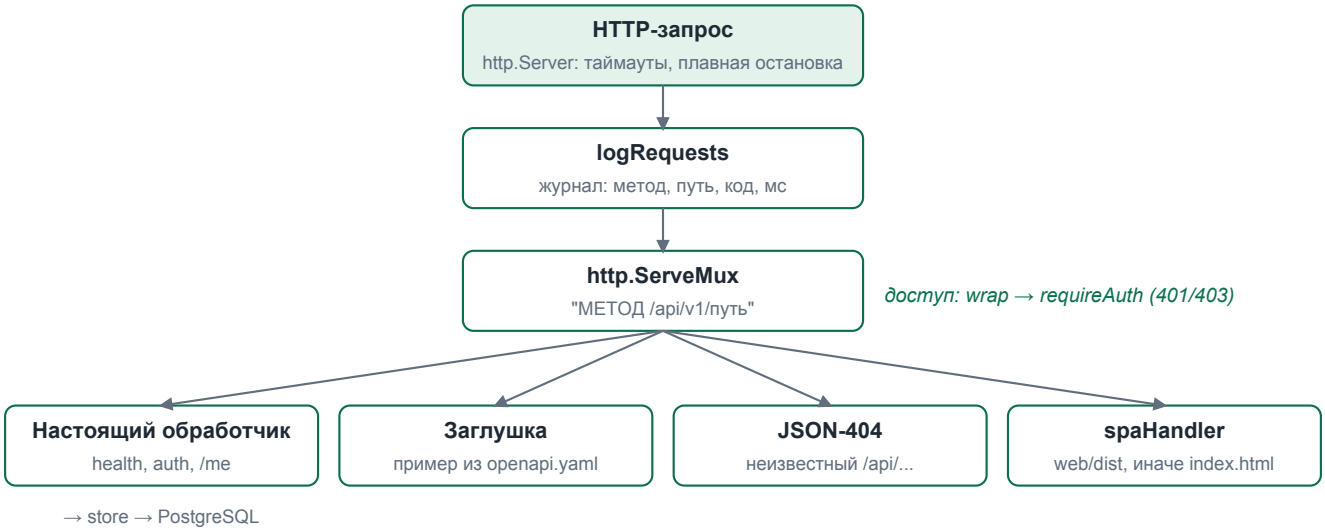
3. Карта файлов

Файл	Строк	За что отвечает
`cmd/server/main.go`	98	Настройки, подключение к базе, миграции, сборка зависимостей, http.Server, остановка по SIGTERM
`api/spec.go`	23	Встраивает openapi.yaml, `Load()` — разбор и проверка описания, `BasePath`
`api/types.gen.go`	—	Типы запросов и ответов из описания (создаётся, не в git)
`migrations/migrations.go`	24	Встроенные *.sql и `Up()` через goose
`internal/httpapi/api.go`	99	Config, маршруты, уровни доступа, `requireAuth`, `UserID(ctx)`
`internal/httpapi/http.go`	104	Единый вид ошибок, `readJSON` с ограничением 1 МБ, раздача SPA, журнал запросов
`internal/httpapi/stubs.go`	76	Заглушки для всех адресов описания без настоящего обработчика
`internal/httpapi/handlers_auth.go`	78	`health`, `register`, `login`
`internal/httpapi/handlers_me.go`	85	`getMe`, `patchMe`, проверка полей профиля
`internal/auth/service.go`	120	Регистрация и вход: проверки, bcrypt, защита от перебора почт
`internal/auth/tokens.go`	74	Выдача и проверка JWT
`internal/store/store.go`	79	Store, ошибки, пользователи (создание, поиск по почте)

Файл	Строк	За что отвечает
`internal/store/profiles.go`	59	Чтение и изменение профиля

Зависимости идут в одну сторону: `main → httpapi → auth → store → api`. Пакет `httpapi` зависит от базы только через интерфейс `Store` (в тестах — подделка).

4. Путь запроса



- Маршрут регистрируется строкой вида `"GET /api/v1/me"` (стандартный ServeMux Go 1.22+): метод и путь проверяет сам маршрутизатор.
- `New()` сначала регистрирует настоящие обработчики (`a.handle`), запоминая их в `a.real`, затем обходит описание и ставит заглушки только туда, где обработчика нет.
- Всё под `/api/v1/`, что не нашлось, — JSON-ошибка `not_found`, а не index.html.
- Остальные пути отдаёт `spaHandler`: существующий файл — как есть, иначе `index.html` с `Cache-Control: no-cache`. Не GET/HEAD — `405 method_not_allowed`.

5. Доступ и токены

Уровень	Как задаётся	Что проверяется
`public`	в коде; для заглушек — `security: []` в описании	ничего
`user`	общий `security` описания	заголовок `Authorization: Bearer <токен>`, подпись, срок → иначе 401
`admin`	`x-admin: true` у операции	то же + признак администратора в токене → иначе 403

- Токен: HS256, поля `sub` (номер пользователя), `adm` (администратор), `iat`, `exp` = +24 ч. Продления нет.
- Проверка разрешает только HS256 (`alg=none` и чужие алгоритмы отклоняются), срок обязателен.
- «Закрыто при сбое»: любая ошибка разбора токена — 401 `unauthorized`.
- Обработчик берёт номер пользователя через `UserID(r.Context())` — права сам не проверяет.
- Признак администратора попадает в токен при входе; если его поменяли в базе, он вступит в силу после нового входа.

6. Регистрация и вход

- Почта приводится к нижнему регистру и проверяется (`net/mail` + точка в домене). В базе — уникальный индекс по `lower(email)`.
- Пароль: от 8 символов, не больше 72 байт (предел bcrypt). bcrypt cost 10.
- `CreateUser` в одной транзакции создаёт пользователя и пустой профиль — профиль есть всегда.
- Неверная почта и неверный пароль дают одинаковый ответ 401 `invalid_credentials`. Для неизвестной почты всё равно выполняется сравнение bcrypt с заглушкой, чтобы время ответа не выдавало, кто

зарегистрирован.

- Ответ на успех: `{access_token, token_type: "Bearer", expires_at, user}`; регистрация — 201, вход — 200.

7. Профиль: GET и PATCH /me

- `GET /me` — пользователь и профиль одним запросом (`users JOIN profiles`). Незаполненные `target_title`, `target_grade`, `salary_from` отдаются как `null`. `skills` пока всегда `[]` — таблицы навыков ещё нет.
- `PATCH /me` меняет только переданные поля: `nil` уходит в SQL как NULL, и `COALESCE` оставляет старое значение. Стереть поле в `null` нельзя (так в описании). Изменения `users.full_name` и `profiles` — в одной транзакции. Ответ — профиль после изменения.
- `skill_ids` принимается и пока игнорируется.
- Пользователя нет (удалён, а токен ещё действует) — 404 `not_found`, а не 500.

Поле	Правило (длина — в символах, не байтах)	Текст ошибки
`full_name`	после обрезки пробелов не пустое	Укажите имя
`target_title`	≤ 100	Должность — не длиннее 100 символов
`target_grade`	junior | middle | senior	Уровень: junior, middle или senior
`cities`	≤ 10 штук, каждый ≤ 100	Можно указать не больше 10 городов / Название города — не длиннее 100 символов
`work_formats`	office | remote | hybrid	Формат работы: office, remote или hybrid
`salary_from`	0 … 10 000 000	Зарплата — от 0 до 10 000 000
`tone`	formal | neutral | friendly	Тон: formal, neutral или friendly

Правила — одна таблица в `validatePatchMe`; допустимые значения берутся из констант `api.*`, созданных из описания, поэтому не расходятся с ним. Ошибка — 400 `validation_error` с первым нарушенным правилом.

8. База и правило доступа к данным

Весь SQL — только в `internal/store`. Главное правило:

```
Функция, которая читает или меняет данные пользователя, принимает userID
первым параметром после ctx и ограничивает запрос условием user_id = $N.
Чужая запись выглядит как несуществующая: ErrNotFound -> 404 (не 403).
Исключение: GetUserByEmail при входе (userID ещё неизвестен).
```

Так проверку невозможно забыть: без userID функцию не вызвать, а 404 не выдаёт, что запись существует.

Таблица	Миграция	Назначение
`users`	00001	почта (уникальна без учёта регистра), хеш пароля, имя, `is_admin`
`profiles`	00001	1:1 с users: должность, уровень, города, форматы, зарплата, тон, статус резюме
`jobs`	00002	очередь фоновых задач (collect, normalize, embed, parse_resume) — пока не используется кодом
`sources`	00002	источники вакансий (headhunter)
`collection_runs`	00002	запуски сбора вакансий и их итоги
`vacancy_raw`	00002	сырые ответы источников, уникальны по (источник, внешний id)

Новая миграция — файл `server/migrations/0000N_что.sql` с разделами `-- +goose Up` / `-- +goose Down`. Сервер применит её сам при следующем запуске.

9. Ошибки

Любая ошибка — `{"error": {"code": "...", "message": "..."}}`. `code` — для программы, `message` — для человека, показывается как есть. Подробности ошибок базы пишутся в журнал, наружу уходит общий текст.

HTTP	code	Когда
400	`invalid_json`	тело не разбирается как JSON нужного вида
400	`validation_error`	поле заполнено неверно
401	`unauthorized`	нет токена, он просрочен или подделан
401	`invalid_credentials`	неверная почта или пароль
403	`forbidden`	адрес только для администратора
404	`not_found`	нет записи (или она чужая); неизвестный адрес под /api/
405	`method_not_allowed`	не GET/HEAD к клиентской части
409	`email_taken`	почта уже зарегистрирована
413	`body_too_large`	JSON больше 1 МБ
500	`internal_error`	прочие ошибки (подробности — в журнале)

10. Договор и заглушки

- Заглушка отвечает примером первого успешного ответа (200/201/202/204) и заголовком `X-Stub: true`. Пример кодируется один раз при запуске: ошибка в описании видна сразу, сервер не стартует.
- Доступ у заглушки такой же, какой будет у настоящего обработчика (по `security` и `x-admin`).
- Клиентская часть строит свои типы и заглушки из того же файла (`pnpm run generate` в `web/`).
- Тесты описания (`api/spec_test.go`): файл проходит проверку OpenAPI, в нём есть все 24 обязательных адреса, у каждого ответа с телом есть пример, у каждого адреса перечислены ошибки.

11. Тесты

Файл	Что проверяет	База
`auth/tokens_test.go`	токен туда-обратно; отклоняются просроченный, чужой, изменённый, alg=none; короткий секрет; формат почты	нет
`httpapi/httpapi_test.go`	health; 401 на плохие токены и UserID в обработчике; 403 для admin; все адреса отвечают; пример заглушки; JSON-404; SPA	нет
`httpapi/auth_db_test.go`	регистрация, вход, одинаковый ответ для неизвестной почты, пустой профиль; PATCH /me: частичное изменение, изоляция пользователей, все ошибки проверки, русские названия; 404 для удалённого пользователя	да
`api/spec_test.go`	описание целое и полное (см. п. 10)	нет

Тесты с базой запускаются при заданном `TEST_DATABASE_URL` (в CI — всегда) и очищают таблицу `users`. Не направляйте их на базу с нужными данными.

CI (`.github/workflows/server.yml`): `make generate` → gofmt → go vet → миграции на пустой базе → `go test -race ./...`.

12. Как добавить настоящий обработчик

1. Убедиться, что адрес и схемы есть в `openapi.yaml`; если меняли описание — `make generate`.
2. SQL — новая функция в `internal/store` с `userID` первым параметром и `user_id = $N` в запросе.
3. Добавить метод в интерфейс `Store` (`httpapi/api.go`) и в подделку `fakeStore` в тестах.
4. Обработчик — метод `*API`: `readJSON` → проверка → вызов store с `UserID(ctx)` → `writeJSON`; ошибки store — через `a.storeError`.
5. Одна строка в `New()`: `a.handle("GET", "/vacancies", user, a.listVacancies)`. Заглушка исчезнет сама.
6. Тест с базой: успех, ошибки проверки и то, что чужие данные не видны (404).

13. Что изменилось при сокращении

	Было	Стало
Рабочий код (руками)	1004 строки, 22 файла	919 строк, 12 файлов
Генерированный код в git	881	0 (создаётся при сборке)
`back_jr/` (не собирался)	214	встроен: `GET/PATCH /me` работают
Дубликат `.github/workflows/Validation.go`	53	удалён
Пустые пакеты (`doc.go`)	9	удалены
Тесты	559	617 (добавлены тесты /me)
Комментарии в коде	были	удалены (кроме `//go:embed`, `//go:generate`, `-- +goose`)
Настоящих адресов	3	5

При переносе `back_jr` исправлено: `int` → `int64`; 404 вместо 500 для отсутствующего пользователя; `null` вместо `""`/`0`; обязательное `skills`; `full_name` в PATCH; ограничение размера тела и код `invalid_json`; длина в символах; верный текст ошибки для города; проверка таблицей правил.

14. Словарь терминов

Все слова, которые встречаются выше, — по темам. Внутри темы — в порядке появления в тексте.

Общие понятия

Термин	Что значит
Бэкенд, сервер	Программа, которая работает на компьютере-сервере: принимает запросы, проверяет права, читает и пишет базу, отвечает. Здесь — Go-программа из папки `server/`.
Клиентская часть (фронтенд)	То, что работает в браузере пользователя: страницы, кнопки, формы. Лежит в `web/`, после сборки — в `web/dist`.
SPA	Single Page Application — «одностраничное приложение»: браузер один раз загружает `index.html` и скрипты, дальше страницы меняются без перезагрузки, а данные приходят через API.
API	Application Programming Interface — набор адресов, через которые программы общаются друг с другом. Клиентская часть ходит к серверу через API.
REST	Стиль построения API: каждый ресурс — свой адрес (`/me`, `/vacancies`), действие задаётся методом HTTP, данные — в JSON.
Адрес (эндпоинт, маршрут)	Сочетание метода и пути, например `GET /api/v1/me`. На каждый адрес сервер отвечает своим обработчиком.
HTTP	Протокол, по которому браузер и сервер обмениваются запросами и ответами.
Метод HTTP	`GET` — получить, `POST` — создать или выполнить действие, `PATCH` — частично изменить, `HEAD` — как GET, но без тела ответа.
Код ответа (статус)	Число в ответе: 2xx — успех (200 «ок», 201 «создано», 204 «нет тела»), 4xx — ошибка клиента (400, 401, 403, 404…), 5xx — ошибка сервера (500).
Заголовок	Служебная строка запроса или ответа вида `Имя: значение`, например `Authorization` или `Cache-Control`.
Тело запроса / ответа	Основное содержимое после заголовков. У нас это почти всегда JSON.
JSON	Текстовый формат данных: `{"email": "a@b.ru", "skills": []}`. `null` в JSON означает «значения нет».
Исполняемый файл	Готовая программа, которую можно запустить. Go собирает весь сервер в один такой файл.
Журнал (лог)	Записи о том, что делает сервер: какие пришли запросы, сколько длились, какие были ошибки. Читают разработчики, пользователи не видят.
Порт, `:8080`	Номер «двери», на которой сервер ждёт запросы. `:8080` — порт 8080 на всех сетевых адресах; `localhost` — свой компьютер.

Язык Go и устройство кода

Термин	Что значит
Go	Язык программирования, на котором написан сервер. Компилируется в один исполняемый файл.
Модуль, `go.mod`	Проект Go целиком: имя и список сторонних библиотек с версиями (`go.sum` — их контрольные суммы).
Пакет	Папка с файлами Go — единица кода (`httpapi`, `auth`, `store`). Пакеты в `internal/` нельзя подключить из других проектов.
`cmd/server/main.go`	Точка входа: с функции `main` программа начинает работу.
Структура, метод	Структура — набор полей (например `API` или `Config`). Метод — функция, привязанная к структуре: `a.patchMe` — метод `*API`.
Интерфейс	Список методов без реализации. `Store` — интерфейс: обработчикам всё равно, настоящая это база или подделка в тестах, лишь бы методы были.
Зависимости	Кто кого использует. «В одну сторону» значит, что нижние пакеты (store) ничего не знают о верхних (httpapi).
Контекст, `ctx`	Объект `context.Context`, который идёт с запросом через все функции: несёт номер пользователя и отменяет работу, если клиент ушёл.
Обработчик (handler)	Функция, которая отвечает на один адрес: читает запрос, делает работу, пишет ответ.
Маршрутизатор, `ServeMux`	Часть стандартной библиотеки Go, которая по методу и пути выбирает нужный обработчик.
Обёртка (middleware)	Функция вокруг обработчика, которая делает общую работу до или после него: `logRequests` пишет журнал, `requireAuth` проверяет токен, `wrap` навешивает нужный уровень доступа.
`nil`	«Ничего» в Go: пустой указатель. В PATCH поле, которое не прислали, равно `nil`.
`int` / `int64`	Целые числа. `int64` — ровно 64 бита на любой машине; номера записей в базе (`bigserial`) такого же размера.
Ошибка-значение (`ErrNotFound`)	Заранее объявленная ошибка, которую можно узнать по имени и превратить в нужный код ответа (здесь — в 404).
`//go:embed`	Указание компилятору вложить файл внутрь программы. Так `openapi.yaml` и миграции `*.sql` всегда едут вместе с сервером.
`go generate`	Запуск команд, указанных строкой `//go:generate` в коде. У нас — создание `types.gen.go`.
Генерированный код	Код, который пишет программа, а не человек (`types.gen.go`). Руками не правят — пересоздают.
`gofmt`	Стандартное форматирование кода Go: отступы и пробелы у всех одинаковые.
`go vet`	Проверка кода на подозрительные места, которые компилятор пропускает.
`go test -race`	Запуск тестов с поиском гонок — ошибок, когда два потока одновременно меняют одни данные.
Таймауты	Предельное время на чтение запроса и запись ответа. Не дают медленному или зависшему клиенту держать сервер вечно.
SIGTERM, плавная остановка	SIGTERM — сигнал операционной системы «завершись». Сервер перестаёт принимать новые запросы, дожидается текущих и только потом выходит.
Флаг `-migrate-only`	Параметр запуска: применить изменения схемы базы и сразу выйти, сервер не поднимать.

Договор API

Термин	Что значит
OpenAPI	Стандартный формат описания REST API: какие есть адреса, что они принимают, что отвечают, какие бывают ошибки.
Договор, описание, `openapi.yaml`	Наш файл в формате OpenAPI. Главный источник правды: по нему строятся типы сервера, типы клиента и заглушки.
YAML	Текстовый формат с отступами вместо скобок. В нём написан `openapi.yaml`.
Схема	Описание формы данных в договоре: какие поля, какого типа, какие обязательны, какие значения допустимы.
Пример (example)	Готовый образец ответа в договоре. Его отдаёт заглушка.
`oapi-codegen`	Программа, которая из `openapi.yaml` создаёт типы Go (`api/types.gen.go`) и константы допустимых значений `api.*`.
Заглушка (stub)	Временный обработчик, который вместо настоящей работы отдаёт пример из договора. Нужен, чтобы клиентская часть могла работать раньше, чем готов сервер.
`X-Stub: true`	Заголовок, по которому видно, что ответ — от заглушки, а не настоящий.
`security`	Раздел договора: нужен ли адресу токен. `security: []` — адрес открыт всем.
`x-admin`	Наша пометка в договоре (`x-` — собственное расширение): адрес только для администратора.
`BasePath`	Общее начало всех адресов API — `/api/v1`. `v1` — первая версия.

База данных

Термин	Что значит
PostgreSQL	Сервер базы данных: хранит таблицы и выполняет SQL-запросы.
pgvector	Расширение PostgreSQL для хранения векторов (наборов чисел) и поиска похожих — пригодится для подбора вакансий по смыслу.
SQL	Язык запросов к базе: `SELECT` — прочитать, `INSERT` — добавить, `UPDATE` — изменить, `DELETE` — удалить.
Таблица, строка (запись), столбец	Таблица — как лист Excel: строки — записи (один пользователь), столбцы — поля (почта, имя).
Схема базы	Какие есть таблицы, столбцы, типы и ограничения.
Миграция	Файл с изменением схемы базы. Миграции применяются по порядку номеров (00001, 00002…), каждая — один раз.
goose	Библиотека, которая применяет миграции и помнит, какие уже применены. `-- +goose Up` — что сделать, `-- +goose Down` — как откатить.
Первичный ключ (PRIMARY KEY)	Столбец, однозначно задающий запись, — обычно `id`. `bigserial` — номер, который база выдаёт сама по порядку.
Внешний ключ (REFERENCES)	Ссылка на запись другой таблицы. `ON DELETE CASCADE` — при удалении пользователя удаляется и его профиль.
1:1	Связь «один к одному»: у каждого пользователя ровно один профиль.
Индекс	Вспомогательная структура для быстрого поиска, как алфавитный указатель в книге. Уникальный индекс ещё и запрещает повторы.
`lower(email)`	Почта в нижнем регистре. Индекс по ней не даёт завести `Ivan@mail.ru` и `ivan@mail.ru` как двух людей.
Транзакция	Группа изменений, которая выполняется целиком или не выполняется вовсе. Пользователь без профиля появиться не может.
`JOIN`	Соединение двух таблиц в одном запросе по общему полю (`users.id = profiles.user_id`).
`NULL`	«Значения нет» в базе. Не то же самое, что пустая строка или 0.
`COALESCE(a, b)`	Берёт первое значение, которое не NULL. `COALESCE($новое, старое)` оставляет старое, если новое не прислали.
`$N`	Место для параметра в SQL (`$1`, `$2`). Значение передаётся отдельно, поэтому его нельзя подставить как кусок запроса (защита от SQL-инъекций).
`text[]`	Массив строк в одном столбце, например список городов.
`jsonb`	Столбец с JSON внутри; база умеет искать по его содержимому.
`timestamptz`	Дата и время с часовым поясом.
`CHECK`	Ограничение в базе: например, уровень может быть только junior, middle или senior.
Store (хранилище)	Наш слой работы с базой (`internal/store`). Весь SQL — только там.
Очередь задач (`jobs`)	Таблица фоновых работ, которые выполняются позже: `collect` — собрать вакансии, `normalize` — привести к единому виду, `embed` — посчитать вектор смысла, `parse_resume` — разобрать резюме.
Сырые данные (`vacancy_raw`)	Ответ источника (например HeadHunter) как есть, до обработки.

Вход и безопасность

Термин	Что значит
Аутентификация / авторизация	Аутентификация — узнать, кто пришёл (вход по паролю, проверка токена). Авторизация — решить, что ему можно (пользователь или администратор).
Токен	Строка, которую сервер выдаёт после входа. Клиент присылает её с каждым запросом вместо пароля.
JWT	JSON Web Token — токен из трёх частей: заголовок, данные (кто, до когда) и подпись. Данные видны всем, но подделать их без секрета нельзя.
Подпись, секрет	Подпись — контрольная сумма данных токена, посчитанная с секретом. Секрет (`JWT_SECRET`) знает только сервер.
HS256	Способ подписи JWT: HMAC с SHA-256 и одним общим секретом.
`alg=none`	Токен «без подписи». Известная атака: если сервер такое принимает, токен может подделать кто угодно. У нас отклоняется.
`sub`	Поле JWT «subject» — номер пользователя.
`adm`	Наше поле JWT: администратор ли пользователь.
`iat` / `exp`	Время выдачи (issued at) и время окончания действия (expires) токена.
Bearer	Тип токена «предъявительский»: кто его прислал, тот и считается владельцем. Отправляется так: `Authorization: Bearer <токен>`.
Хеш пароля	Результат необратимого преобразования пароля. В базе хранится только он: сам пароль восстановить нельзя, но можно проверить, совпадает ли введённый.
bcrypt	Алгоритм хеширования паролей, специально медленный, чтобы пароли нельзя было быстро подбирать. Обрабатывает не больше 72 байт пароля.
cost	Сложность bcrypt: 10 — 2^{10} повторов внутри. Больше — медленнее и надёжнее.
Байт и символ	Латинская буква — 1 байт, русская — 2 байта. Поэтому длины полей мы считаем в символах, а предел bcrypt — в байтах.
Перебор почт	Попытка узнать, зарегистрирован ли человек, по разнице в ответе или во времени ответа. Защита: одинаковый ответ и сравнение bcrypt даже для неизвестной почты.
«Закрыто при сбое»	Если при проверке что-то пошло не так, доступ запрещается, а не разрешается.
401 и 403	401 — «не знаю, кто вы» (нет токена или он плохой). 403 — «знаю, но нельзя» (нужен администратор).
404 вместо 403 для чужого	Чужая запись выглядит несуществующей — так нельзя даже узнать, что она есть.
Изоляция пользователей	Каждый видит и меняет только свои данные. Проверяется тестами.

Сборка, запуск, проверки

Термин	Что значит
Переменная окружения	Настройка, которую программа получает от системы при запуске (`DATABASE_URL`, `JWT_SECRET`…). Пароли и секреты не хранят в коде.
`DATABASE_URL`	Адрес базы в виде `postgres://пользователь:пароль@хост:порт/база`. `sslmode=disable` — без шифрования, только для своей машины.
Docker, образ, контейнер	Docker упаковывает программу со всем окружением. Образ — упаковка, контейнер — запущенный образ.
`Dockerfile`	Рецепт сборки образа. У нас в два этапа: сначала собирается программа, потом она одна кладётся в маленький образ alpine.
`docker compose`	Запуск нескольких контейнеров разом по файлу `docker-compose.yml`: `db` — база, `ml` — сервис машинного обучения, `server` — сервер.
`HEALTHCHECK`, `/health`	Регулярная проверка «жив ли сервер»: Docker обращается к `/api/v1/health` и при сбоях считает контейнер больным.
`make`, `Makefile`	Короткие команды для длинных действий: `make test` вместо полной строки `go test` с настройками.
git, клон	git хранит историю изменений. Клон — копия репозитория на своём компьютере.
CI	Continuous Integration — автоматическая проверка каждого изменения на сервере GitHub (GitHub Actions): сборка, форматирование, тесты.
IDE	Редактор кода с подсказками (VS Code, GoLand). Без `types.gen.go` он будет подчёркивать ошибки.
Тест	Код, который запускает рабочий код и проверяет результат. Тесты без базы быстрые; с базой — проверяют настоящий SQL.
Подделка (fake)	Ненастоящая реализация интерфейса для тестов. `fakeStore` хранит данные в памяти вместо базы.
`TEST_DATABASE_URL`	Адрес отдельной базы для тестов. Если не задан, тесты с базой пропускаются.
pnpm	Менеджер пакетов JavaScript; `pnpm run generate` создаёт типы клиентской части из того же `openapi.yaml`.
`index.html`	Главная страница SPA. На любой неизвестный путь сервер отдаёт её, а нужную страницу выбирает уже браузер.
`Cache-Control: no-cache`	Заголовок «перед использованием уточни у сервера, не изменилось ли». Нужен, чтобы после обновления не застревала старая версия страницы.